

BÁO CÁO BÀI LAB 01: CẤU HÌNH KERNEL & KHỞI ĐỘNG HỆ THỐNG NHÚNG

Mục 1: Thông tin sinh viên

Họ và tên: Trần Nguyễn Việt Dũng

Mã số sinh viên (MSSV): SE205180

Nhóm:

Ngày thực hiện: 15/09/2026

Mục 2: Boot Sequence Diagram

SƠ ĐỒ CHUỖI KHỞI ĐỘNG (BOOT SEQUENCE DIAGRAM)



Quá trình khởi động trải qua 6 giai đoạn chính: Cấp nguồn -> Mask ROM nạp U-Boot vào RAM -> U-Boot truyền tham số và nạp Linux Kernel -> Kernel mount RootFS từ RAM (**initramfs**) -> Gọi tiến trình **PID 1** (/sbin/init) đọc **inittab** -> Mở giao diện BusyBox Shell.

Mục 3: Log thực hành

Phần A: Build Linux Kernel

```
tran-dung@tran-dung-G5-KC: ~/embedded_lab1/kern...
~/embedded_lab1/kernel/linux-5.15

GZIP arch/arm/boot/compressed/piggy_data
CC arch/arm/boot/compressed/misc.o
CC arch/arm/boot/compressed/decompress.o
CC arch/arm/boot/compressed/string.o
SHIPPED arch/arm/boot/compressed/hyp-stub.S
CC arch/arm/boot/compressed/fdt_rw.o
CC arch/arm/boot/compressed/fdt_ro.o
CC arch/arm/boot/compressed/fdt_wip.o
CC arch/arm/boot/compressed/fdt.o
CC arch/arm/boot/compressed/fdt_check_mem_start.o
SHIPPED arch/arm/boot/compressed/libifuncs.S
SHIPPED arch/arm/boot/compressed/ashldi3.S
SHIPPED arch/arm/boot/compressed/bswapsdi2.S
AS arch/arm/boot/compressed/hyp-stub.o
AS arch/arm/boot/compressed/libifuncs.o
AS arch/arm/boot/compressed/ashldi3.o
AS arch/arm/boot/compressed/bswapsdi2.o
AS arch/arm/boot/compressed/piggy.o
LD arch/arm/boot/compressed/vmlinux
/usr/bin/arm-linux-gnueabi-hf-ld.bfd: warning: arch/arm/boot/compressed/vmlinux
as a LOAD segment with RWX permissions
OBJCOPY arch/arm/boot/zImage
Kernel: arch/arm/boot/zImage is ready
tran-dung@tran-dung-G5-KC:~/embedded_lab1/kernel/linux-5.15$
```

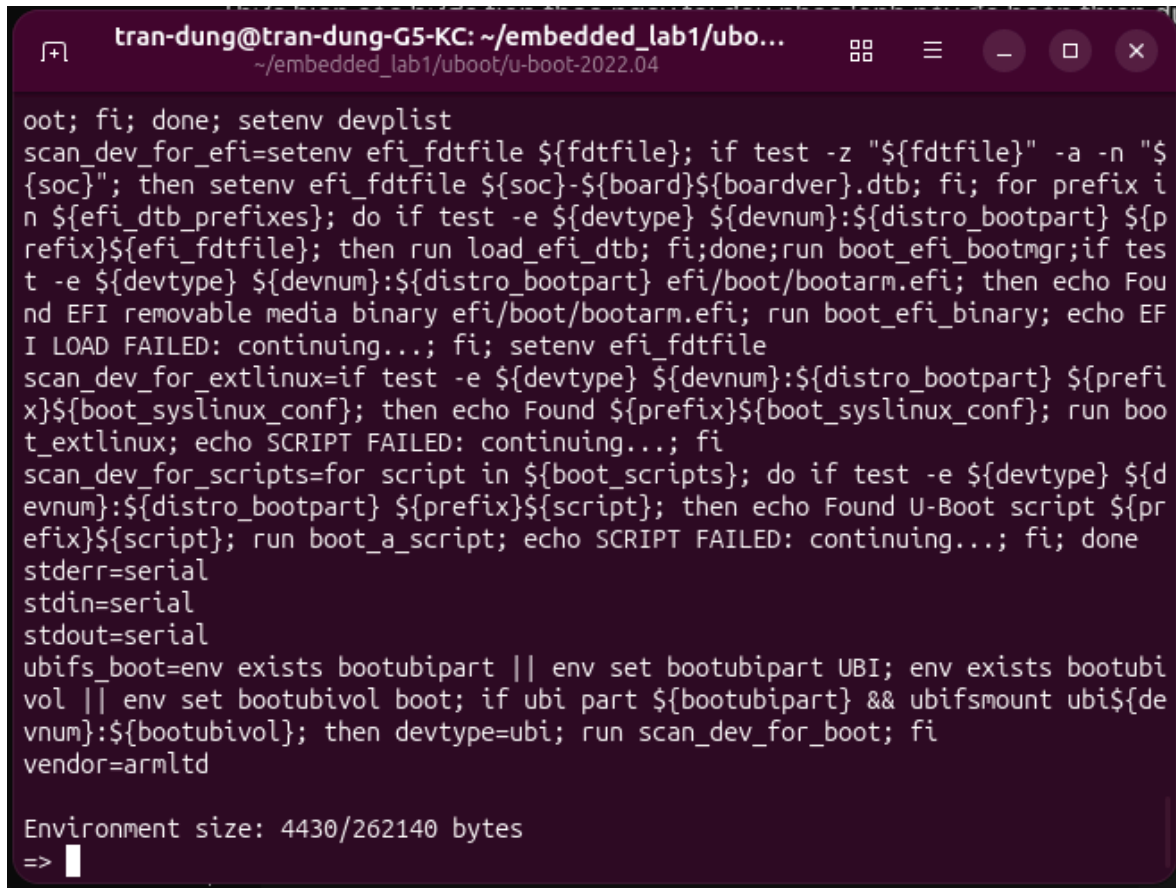
Hình 1: Khai báo cross-compile toolchain **arm-linux-gnueabihf** và khởi tạo cấu hình kernel mặc định cho bo mạch **vexpress-a9**.

```
tran-dung@tran-dung-G5-KC: ~/embedded_lab1/kern...  
~/embedded_lab1/kernel/linux-5.15  
SHIPPED arch/arm/boot/compressed/bswapsdi2.S  
AS      arch/arm/boot/compressed/hyp-stub.o  
AS      arch/arm/boot/compressed/lib1funcs.o  
AS      arch/arm/boot/compressed/ashldi3.o  
AS      arch/arm/boot/compressed/bswapsdi2.o  
AS      arch/arm/boot/compressed/piggy.o  
LD      arch/arm/boot/compressed/vmlinux  
/usr/bin/arm-linux-gnueabihf-ld.bfd: warning: arch/arm/boot/compressed/vmlinux  
as a LOAD segment with RWX permissions  
OBJCOPY arch/arm/boot/zImage  
Kernel: arch/arm/boot/zImage is ready  
tran-dung@tran-dung-G5-KC:~/embedded_lab1/kernel/linux-5.15$ ls -lh arch/arm/bo  
t/zImage arch/arm/boot/dts/vexpress-v2p-ca9.dtb  
file arch/arm/boot/zImage  
arm-linux-gnueabihf-readelf -A arch/arm/boot/compressed/vmlinux | grep -i 'Tag_  
PU_arch'  
-rw-rw-r-- 1 tran-dung tran-dung 14K Sep 17 21:22 arch/arm/boot/dts/vexpress-v  
p-ca9.dtb  
-rwxrwxr-x 1 tran-dung tran-dung 4.9M Sep 17 21:24 arch/arm/boot/zImage  
arch/arm/boot/zImage: Linux kernel ARM boot executable zImage (kernel >=v4.15)  
little-endian)  
Tag_CPU_arch: v7  
Tag_CPU_arch_profile: Application  
tran-dung@tran-dung-G5-KC:~/embedded_lab1/kernel/linux-5.15$
```

Hình 2: Xác minh kết quả build Kernel ARMv7 (kích thước file zImage, Device Tree và thông tin kiến trúc v7 bằng readelf).

```
tran-dung@tran-dung-G5-KC: ~/embedded_lab1/ubo...  
~/embedded_lab1/uboot/u-boot-2022.04  
flashoffset = 0x00000000  
baudrate    = 38400 bps  
relocaddr   = 0x7ff6c000  
reloc off   = 0x1f76c000  
Build       = 32-bit  
current eth = ethernet@3,02000000  
ethaddr     = 52:54:00:12:34:56  
IP_addr     = <NULL>  
fdt_blob    = 0x7ffe0430  
new_fdt     = 0x00000000  
fdt_size    = 0x00000000  
lmb_dump_all:  
memory.cnt  = 0x1  
memory[0]   = [0x60000000-0x80000003], 0x20000004 bytes flags: 0  
reserved.cnt = 0x2  
reserved[0] = [0x4c000000-0x4c7fffff], 0x00800000 bytes flags: 4  
reserved[1] = [0x7fb27c88-0x7fffffff], 0x004d8378 bytes flags: 0  
devicetree  = embed  
arch_number = 0x000008e0  
TLB_addr    = 0x7fff0000  
irq_sp      = 0x7fb2bec0  
sp_start    = 0x7fb2beb0  
Early malloc usage: 370 / 400  
=> printenv
```

Phần B: Cấu hình và Sử dụng U-Boot



```
tran-dung@tran-dung-G5-KC: ~/embedded_lab1/ubo...  
~/embedded_lab1/u-boot/u-boot-2022.04  
oot; fi; done; setenv devplist  
scan_dev_for_efi=setenv efi_fdtfile ${fdtfile}; if test -z "${fdtfile}" -a -n "${  
{soc}"; then setenv efi_fdtfile ${soc}-${board}${boardver}.dtb; fi; for prefix i  
n ${efi_dtb_prefixes}; do if test -e ${devtype} ${devnum}:${distro_bootpart} ${p  
refix}${efi_fdtfile}; then run load_efi_dtb; fi;done;run boot_efi_bootmgr;if tes  
t -e ${devtype} ${devnum}:${distro_bootpart} efi/boot/bootarm.efi; then echo Fou  
nd EFI removable media binary efi/boot/bootarm.efi; run boot_efi_binary; echo EF  
I LOAD FAILED: continuing...; fi; setenv efi_fdtfile  
scan_dev_for_extlinux=if test -e ${devtype} ${devnum}:${distro_bootpart} ${prefi  
x}${boot_syslinux_conf}; then echo Found ${prefix}${boot_syslinux_conf}; run boo  
t_extlinux; echo SCRIPT FAILED: continuing...; fi  
scan_dev_for_scripts=for script in ${boot_scripts}; do if test -e ${devtype} ${d  
evnum}:${distro_bootpart} ${prefix}${script}; then echo Found U-Boot script ${pr  
efix}${script}; run boot_a_script; echo SCRIPT FAILED: continuing...; fi; done  
stderr=serial  
stdin=serial  
stdout=serial  
ubifs_boot=env exists bootubipart || env set bootubipart UBI; env exists bootubi  
vol || env set bootubivol boot; if ubi part ${bootubipart} && ubifsmount ubi${de  
vnum}:${bootubivol}; then devtype=ubi; run scan_dev_for_boot; fi  
vendor=arm ltd  
  
Environment size: 4430/262140 bytes  
=> |
```

Hình 4: Danh sách biến môi trường mặc định của U-Boot trên board vexpress-a9).



```
tran-dung@tran-dung-G5-KC: ~/embedded_lab1/ubo...  
~/embedded_lab1/u-boot/u-boot-2022.04  
nd EFI removable media binary efi/boot/bootarm.efi; run boot_efi_binary; echo EF  
I LOAD FAILED: continuing...; fi; setenv efi_fdtfile  
scan_dev_for_extlinux=if test -e ${devtype} ${devnum}:${distro_bootpart} ${prefi  
x}${boot_syslinux_conf}; then echo Found ${prefix}${boot_syslinux_conf}; run boo  
t_extlinux; echo SCRIPT FAILED: continuing...; fi  
scan_dev_for_scripts=for script in ${boot_scripts}; do if test -e ${devtype} ${d  
evnum}:${distro_bootpart} ${prefix}${script}; then echo Found U-Boot script ${pr  
efix}${script}; run boot_a_script; echo SCRIPT FAILED: continuing...; fi; done  
stderr=serial  
stdin=serial  
stdout=serial  
ubifs_boot=env exists bootubipart || env set bootubipart UBI; env exists bootubi  
vol || env set bootubivol boot; if ubi part ${bootubipart} && ubifsmount ubi${de  
vnum}:${bootubivol}; then devtype=ubi; run scan_dev_for_boot; fi  
vendor=arm ltd  
  
Environment size: 4430/262140 bytes  
=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem  
=512M'  
=> setenv bootcmd 'bootz 0x60100000 0x64000000 0x60000000'  
=> printenv bootargs  
bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M  
=> printenv bootcmd  
bootcmd=bootz 0x60100000 0x64000000 0x60000000  
=>
```

Hình 5: Thiết lập biến bootargs và bootcmd trên U-Boot).

```
tran-dung@tran-dung-G5-KC: ~/embedded_lab1/ubo...  
~/embedded_lab1/u-boot/u-boot-2022.04  
VFS: Cannot open root device "ram0" or unknown-block(0,0): error -6  
Please append a correct "root=" boot option; here are the available partitions:  
1f00          131072 mtdblock0  
(driver?)  
1f01          32768 mtdblock1  
(driver?)  
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)  
CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.15.0 #1  
Hardware name: ARM-Versatile Express  
[<801103a0>] (unwind_backtrace) from [<8010c2c4>] (show_stack+0x10/0x14)  
[<8010c2c4>] (show_stack) from [<80867f00>] (dump_stack_lvl+0x40/0x4c)  
[<80867f00>] (dump_stack_lvl) from [<80863f64>] (panic+0xfc/0x2e4)  
[<80863f64>] (panic) from [<80b01714>] (mount_block_root+0x1bc/0x264)  
[<80b01714>] (mount_block_root) from [<80b01b98>] (prepare_namespace+0x148/0x184)  
)  
[<80b01b98>] (prepare_namespace) from [<8086c6b8>] (kernel_init+0x14/0x13c)  
[<8086c6b8>] (kernel_init) from [<80100130>] (ret_from_fork+0x14/0x24)  
Exception stack(0x810a7fb0 to 0x810a7ff8)  
7fa0: 00000000 00000000 00000000 00000000  
7fc0: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000  
7fe0: 00000000 00000000 00000000 00000000 00000013 00000000  
---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-blo  
ck(0,0) ]---
```

Hình 6: Lỗi Kernel panic xảy ra khi khởi động Kernel mà chưa nạp RootFS (initramfs) vào bộ nhớ RAM.

Phần C: Xây dựng Root Filesystem và initramfsS

```
tran-dung@tran-dung-G5-KC:~/embedded_lab1/rootfs$ ls -la  
total 40  
drwxrwxr-x 10 tran-dung tran-dung 4096 Sep 15 07:40 .  
drwxrwxr-x 7 tran-dung tran-dung 4096 Sep 15 07:37 ..  
drwxrwxr-x 2 tran-dung tran-dung 4096 Sep 15 07:37 bin  
drwxrwxr-x 2 tran-dung tran-dung 4096 Sep 15 07:40 dev  
drwxrwxr-x 3 tran-dung tran-dung 4096 Sep 15 07:40 etc  
drwxrwxr-x 2 tran-dung tran-dung 4096 Sep 15 07:40 lib  
lrwxrwxrwx 1 tran-dung tran-dung 11 Sep 15 07:37 linuxrc -> bin/busybox  
drwxrwxr-x 2 tran-dung tran-dung 4096 Sep 15 07:40 proc  
drwxrwxr-x 2 tran-dung tran-dung 4096 Sep 15 07:37/sbin  
drwxrwxr-x 2 tran-dung tran-dung 4096 Sep 15 07:40 sys  
drwxrwxr-x 4 tran-dung tran-dung 4096 Sep 15 07:37 usr
```

Hình 7: Kết quả kiểm tra cấu trúc cây thư mục RootFS (Output của lệnh `ls initramfs/` hoặc `ls rootfs/`)

Đóng gói RootFS: Sử dụng công cụ `cpio` với chuẩn định dạng `newc` kết hợp bộ nén `gzip -9` để nén toàn bộ bộ thư mục hệ thống thành file nén `initramfs.cpio.gz`. Kết quả thu được file nén tối ưu có dung lượng **8.0 MB**, sẵn sàng để Kernel nạp vào RAM khi boot.

```
tran-dung@tran-dung-G5-KC:~/embedded_lab1$ cd rootfs
find . | cpio -o -H newc | gzip -9 > ../initramfs.cpio.gz
cd ..
40094 blocks
tran-dung@tran-dung-G5-KC:~/embedded_lab1$ ls -lh initramfs.cpio.gz
-rw-rw-r-- 1 tran-dung tran-dung 8.0M Sep 15 08:06 initramfs.cpio.gz
tran-dung@tran-dung-G5-KC:~/embedded_lab1$
```

Hình 8: Kết quả đóng gói thành công file `initramfs.cpio.gz` (Dung lượng 8.0 MB).

```
total 40
drwxrwxr-x 10 tran-dung tran-dung 4096 Sep 15 07:40 .
drwxrwxr-x 10 tran-dung tran-dung 4096 Sep 15 07:40 ..

tran-dung@tran-dung-G5-KC: ~/embedded_lab1 — qe...
~/embedded_lab1

drm-clcd-pl111 10020000.clcd: Using non-panel bridge
[drm] Initialized pl111 1.0.0 20170317 for 10020000.clcd on minor 0
input: AT Raw Set 2 keyboard as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000:iofpga@7,00000000/10006000.kmi/serio0/input/input0
Console: switching to colour frame buffer device 128x48
drm-clcd-pl111 10020000.clcd: [drm] fb0: pl111 frame buffer device
ALSA device list:
 #0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
Freeing unused kernel image (initmem) memory: 1024K
Run /sbin/init as init process
=====
Welcome to Embedded Linux Systems!
=====

Please press Enter to activate this console. input: ImExPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/bus@40000000:motherboard-bus@40000000:iofpga@7,00000000/10007000.kmi/serio1/input/input2

/ # random: fast init done
/ #
/ #
```

Hình 9: Kernel nạp thành công RootFS, thực thi kịch bản `/sbin/init` và in ra thông báo chào mừng.

```
tran-dung@tran-dung-G5-KC: ~/embedded_lab1 — qe...
~/embedded_lab1

/ # random: fast init done
/ #
/ # ls -la
total 8
drwxrwxr-x 12 1000 1000 0 Sep 15 01:24 .
drwxrwxr-x 12 1000 1000 0 Sep 15 01:24 ..
-rw-r--r-- 1 0 0 7 Sep 15 01:24 .ash_history
drwxrwxr-x 2 1000 1000 0 Sep 15 00:37 bin
drwxrwxr-x 2 1000 1000 0 Sep 15 00:40 dev
drwxrwxr-x 3 1000 1000 0 Sep 15 00:40 etc
-rw-rw-r-- 1 1000 1000 266 Sep 15 01:01 initramfs.cpio.gz
drwxrwxr-x 2 1000 1000 0 Sep 15 00:40 lib
lrwxrwxrwx 1 1000 1000 11 Sep 15 00:37 linuxrc -> bin/busybox
dr-xr-xr-x 103 0 0 0 Sep 15 01:24 proc
drwx----- 2 0 0 0 Sep 14 16:28 root
drwxrwxr-x 3 1000 1000 0 Sep 15 01:00 rootfs
drwxrwxr-x 2 1000 1000 0 Sep 15 00:37 sbin
dr-xr-xr-x 12 0 0 0 Sep 15 01:24 sys
drwxrwxr-x 4 1000 1000 0 Sep 15 00:37 usr
/ # uname -a
Linux (none) 5.15.0 #1 SMP Mon Sep 14 23:28:20 +07 2026 armv7l GNU/Linux
/ #
```

Hình 10: Xác minh cấu trúc file system (`ls -la`) và kiến trúc kernel ARMv7 (`uname -a`) chạy trực tiếp trên QEMU.

- **Khởi động trên QEMU:** Hệ thống được giả lập qua board `vexpress-a9`.

Kernel nạp thành công bộ nhớ tạm `initramfs`, thực thi file script

`/etc/init.d/rcS` và in dòng thông báo chào mừng.

- **Xác minh môi trường:** Kết quả kiểm tra bằng lệnh `uname -a` xác nhận Kernel **ARMv7** đang chạy ổn định. Lệnh `ls -la` cho thấy đầy đủ các thư mục ảo `/proc`, `/sys`, `/dev` đã được mount thành công.

Mục 4: Câu hỏi phân tích

Câu 1: So sánh hai toolchain arm-linux-gnueabi và arm-linux-gnueabihf

- **Sự khác biệt về ABI và FPU:** ABI (Application Binary Interface) `gnueabi` sử dụng chuẩn "soft-float", tức là các tham số dấu phẩy động sẽ được truyền thông qua các thanh ghi số nguyên (integer registers) tiêu chuẩn. Ngược lại, `gnueabihf` ("hard-float") cho phép truyền thẳng tham số dấu phẩy động qua các thanh ghi của bộ xử lý toán học (FPU registers).
- **Lý do dùng gnueabihf cho Cortex-A9:** Lõi Cortex-A9 (dùng kiến trúc ARMv7-A) được tích hợp sẵn FPU vật lý (thường là VFPv3). Sử dụng công cụ `gnueabihf` sẽ tận dụng được phần cứng này để tính toán dấu phẩy động một cách trực tiếp, tối ưu hóa hiệu suất truyền dữ liệu hàm thay vì phải đi đường vòng qua bộ vi xử lý số nguyên.
- **Nếu build kernel bằng gnueabi chạy trên Cortex-A9:** Bản thân Linux kernel rất hiếm khi thực hiện phép toán dấu phẩy động, nên kernel build bằng `gnueabi` vẫn sẽ boot thành công. Tuy nhiên, nếu user-space (như thư mục `rootfs` được đóng gói) được build bằng thư viện `gnueabihf` nhưng lại thực thi trên một hệ thống đang dùng chuẩn `gnueabi`, sẽ xảy ra lỗi bất đồng bộ ABI. Các tiến trình phần mềm sẽ bị lỗi (Segmentation fault) do sai lệch cách tổ chức bộ nhớ khi truyền tham số.

Câu 2: Memory map của vexpress-a9 và thiết lập bootcmd

- **Tại sao bắt đầu từ 0x60000000:** Bản đồ bộ nhớ (Memory map) hoàn toàn được quy định bởi kiến trúc thiết kế phần cứng của bo mạch và System-on-Chip (SoC). Với bo mạch giả lập `vexpress-a9`, bộ điều khiển vùng nhớ quy định dải RAM khả dụng bắt đầu từ địa chỉ vật lý `0x60000000`. Các hệ thống đời cũ hơn như Versatile PB lại có RAM map ở địa chỉ `0x00000000`.
- **Ảnh hưởng đến U-Boot:** Khi lập trình biến `bootcmd`, U-Boot phải nạp các thành phần (như kernel, Device Tree, và file bộ nhớ tạm `initramfs`) vào phân vùng RAM hợp lệ. Do đó, trên board `vexpress-a9`, các địa chỉ này bắt buộc phải nằm trong dải `0x60000000` trở lên (ví dụ: Kernel nạp tại `0x60008000`, DTB tại `0x61000000`, `initramfs` tại `0x62000000`).
- **Nếu đặt sai địa chỉ load:** Phần cứng sẽ lập tức sinh ra lỗi Data Abort (vùng nhớ không hợp lệ hoặc chỉ được đọc). Trình giả lập QEMU hoặc thiết bị thật sẽ đóng băng,

kernel không nhận diện được file hệ thống initramfs 8.0 MB, hoặc kernel panic do đọc phải dữ liệu rác.

Câu 3: Phân tích MIDR (Main ID Register) 0x410fc090 của kiến trúc ARMv7 Mã định danh 0x410fc090 (thuộc kiến trúc kernel ARMv7) được phân tách theo quy chuẩn thành ghi ARM:

- **Implementer (0x41):** Là mã ASCII cho chữ 'A', biểu thị nhà thiết kế lõi vi xử lý là ARM Limited.
- **Variant (0x0):** Phiên bản lõi chính (Major revision 0).
- **Architecture (0xf):** Giá trị này chỉ định CPU sử dụng định dạng nhận dạng CPUID mới (từ kiến trúc ARMv4 trở lên).
- **Part number (0xc09):** Mã định danh dòng chip vi xử lý. Tra cứu thực tế mã 0xc09 tương ứng với lõi **ARM Cortex-A9**.
- **Revision (0x0):** Phiên bản tinh chỉnh phụ (Minor revision 0).

Câu 4: Tùy chọn Static và Dynamic linking trong initramfs

- **Lý do build BusyBox static linking:** Trong giai đoạn khởi động tối giản từ file initramfs, môi trường bộ nhớ rất hạn chế. Build tĩnh (static) sẽ nhúng trực tiếp toàn bộ mã nguồn của các thư viện dùng chung vào bên trong file thực thi duy nhất của BusyBox. Quá trình boot gọi tiến trình PID 1 (/sbin/init) sẽ độc lập tuyệt đối, không cần mất công tổ chức hoặc truy xuất thêm hệ thống thư mục /lib.
- **Nếu build dynamic linking cho ARMv7:** Ta bắt buộc phải copy thủ công các file thư viện dạng .so (shared object) vào rootfs để BusyBox gọi được lệnh. 3 file tối thiểu gồm:
 1. **ld-linux-armhf.so.3** (Trình liên kết động - Dynamic Linker).
 2. **libc.so.6** (Thư viện chuẩn ngôn ngữ C).
 3. **libm.so.6** (Thư viện toán học) hoặc **libcrypt.so.1**.
- **Vị trí trong toolchain:** Nằm bên trong thư mục root ảo (sysroot) của bộ biên dịch chéo, cụ thể ở đường dẫn <thư_mục_cài_đặt_toolchain>/arm-linux-gnueabi/hf/libc/lib/ hoặc <thư_mục_cài_đặt_toolchain>/arm-linux-gnueabi/hf/sysroot/lib/.

Câu 5: Giai đoạn khởi tạo Kernel chuẩn SMP (Symmetric Multi-Processing)

- **Khởi tạo thêm cho -smp 2:** Với hệ thống đa nhân, lúc khởi động chỉ có CPU0 (nhân chính) thức dậy. Để vận hành SMP, kernel buộc phải cài đặt thêm bộ quản lý ngắt tổng GIC (Generic Interrupt Controller) để nhận diện các CPU, cấp phát bộ định thời gian (timer) riêng rẽ trên từng nhân, và cuối cùng CPU0 phải gửi các ngắt nội bộ hệ thống IPI (Inter-Processor Interrupts) để đánh thức CPU1 thông qua giao thức boot phần cứng.
- **2 dòng boot log thay đổi và ý nghĩa:**

1. **Dòng log 1: CPU1: Booted secondary processor** (hoặc **Booting secondary CPU 1 up**).
 - *Ý nghĩa:* Kernel trên nhân CPU0 đã gửi thành công tín hiệu đánh thức, CPU thứ 2 (CPU1) đã khởi chạy và bắt đầu tiến trình khởi tạo phần cứng cục bộ.
2. **Dòng log 2: smp: Brought up 1 node, 2 CPUs.**
 - *Ý nghĩa:* Kernel xác nhận sơ đồ cấu trúc mạng đa nhân đã được thiết lập hoàn tất; cả 2 CPU đều trong trạng thái online và sẵn sàng nhận công việc từ hệ điều phối tiến trình (scheduler).

Mục 5: Nhận xét & Khó khăn gặp phải

During the implementation of Lab 01, the following main challenges were encountered and resolved:

- **Lỗi Kernel Panic: Unable to mount root fs on unknown-block(0,0)**
 - *Mô tả lỗi:* Kernel khởi động thành công nhưng bị dừng lại đột ngột kèm thông báo kernel panic do không tìm thấy tiến trình khởi tạo hoặc RootFS.SS
 - *Cách giải quyết:* Kiểm tra lại chuỗi tham số boot command-line trong lệnh QEMU. Sửa lại chính xác tham số **rdinit=/sbin/init** thay vì **init=/sbin/init** (vì hệ thống dùng **initramfs** nạp trên RAM chứ không phải ổ đĩa khối block device).
- **Lỗi kẹt bàn phím không thoát được QEMU ở chế độ -nographic**
 - *Mô tả lỗi:* Khi hệ thống bị panic hoặc chạy xong, gõ phím tắt **Ctrl + A** rồi **X** không phản hồi khiến terminal bị đơ.
 - *Cách giải quyết:* Mở thêm một tab Terminal mới và dùng lệnh **killall -9 qemu-system-arm** để giải phóng tiến trình QEMU, đưa terminal trở lại trạng thái sẵn sàng nhập lệnh.